Flask App Directory Structure — Complete Guide

1. Why Structure Matters

A small Flask app (like the ones you've been building — `app.py` with a few routes) works fine as a single file. But as a project grows to include multiple pages, forms, database models, and static assets, a single file becomes hard to navigate and maintain.

Flask doesn't force any particular structure on you (unlike Django) — but there are well-established conventions the community follows. This guide covers the structure appropriate at each stage of a project's growth.

---

2. Level 1: Beginner / Single-File Structure

When to use it

Small projects, learning exercises, quick prototypes, single-page apps — exactly what you've been building so far (login forms, profile pages, Jinja practice).

```
Python_with_flask/
│
├── venv/                    # virtual environment
├── app.py                   # all routes and logic
├── templates/                # HTML files (Jinja2)
│   ├── login.html
│   ├── Home.html
│   └── profile.html
├── static/                   # CSS, JS, images
│   ├── style.css
│   └── script.js
└── requirements.txt          # pip freeze output
```

Rules Flask enforces here: - `templates/` must be named exactly that — `render_template()` looks there automatically - `static/` must be named exactly that — referenced in HTML via `{{ url_for('static', filename='style.css') }}`

This is exactly the structure you've been using in this project so far, and it's completely fine at this stage.

---

3. Level 2: Small Multi-Page Project Structure

When to use it

Once you have several related pages (forms, profile, dashboard) and want to keep things from feeling cluttered in a single `app.py`, but the project is still small enough not to need a full package structure.

```
flask_project/
│
├── venv/
├── app.py                    # app creation + route imports
├── config.py                  # configuration settings (secret key, DB URI)
├── requirements.txt
│
├── templates/
│   ├── base.html               # shared layout
│   ├── login.html
│   ├── register.html
│   └── profile.html
│
└── static/
    ├── css/
    │   └── style.css
    ├── js/
    │   └── script.js
    └── images/
```

Example `config.py`

```
class Config:
    SECRET_KEY = "my-secret-key"
    DEBUG = True
```

Example `app.py`

```
from flask import Flask
from config import Config

app = Flask(__name__)
app.config.from_object(Config)

from routes import *   # or define routes directly here

if __name__ == "__main__":
    app.run(debug=True)
```

---

4. Level 3: Production-Style Structure (Application Factory + Blueprints)

When to use it

Larger applications — multiple feature areas (auth, blog, dashboard, API), a team working on the codebase, or a project that will eventually need testing, database migrations, and deployment.

This is the structure shown in most professional tutorials and matches what was in your screenshot.

```
flask_app/
│
├── run.py                    # entry point — starts the app
├── requirements.txt
├── .env                       # environment variables (never commit this)
├── .gitignore
│
└── app/                        # the actual application package
    ├── __init__.py             # application factory (create_app)
    ├── config.py                # configuration classes
    │
    ├── models/                    # database models
    │   ├── __init__.py
    │   └── user.py
    │
    ├── routes/                     # blueprints (grouped routes)
    │   ├── __init__.py
    │   ├── auth.py                 # /login, /register, /logout
    │   ├── main.py                  # /, /home
    │   └── profile.py                # /profile
    │
    ├── templates/
    │   ├── base.html
    │   ├── auth/
    │   │   ├── login.html
    │   │   └── register.html
    │   └── main/
    │       └── home.html
    │
    └── static/
        ├── css/
        │   └── style.css
        ├── js/
        │   └── script.js
        └── images/
```

What each part does

Folder/File	Purpose
`run.py`	The single file you run (`python run.py`) — imports and starts the app
`app/__init__.py`	Contains the **application factory** — builds and configures the Flask app
`app/config.py`	Holds configuration classes (Dev, Testing, Production settings)
`app/models/`	Database table definitions (usually with SQLAlchemy)
`app/routes/`	**Blueprints** — each file groups related routes together
`app/templates/`	HTML files, often organized into subfolders matching each blueprint
`app/static/`	CSS, JS, images
`.env`	Stores secrets (API keys, DB passwords) outside of source code

---

5. The Application Factory Pattern — `app/__init__.py`

Description

Instead of creating the Flask app directly in `app.py` , you wrap creation in a function. This makes it easy to create multiple instances of the app (useful for testing) and keeps configuration flexible.

```
from flask import Flask

def create_app():
    app = Flask(__name__)
    app.config.from_object("app.config.Config")

    # register blueprints
    from app.routes.auth import auth_bp
    from app.routes.main import main_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)

    return app
```

run.py

```
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

---

6. Blueprints — Splitting Routes Across Files

Description

A Blueprint lets you define a group of related routes in their own file, then register them with the main app. This is how large apps avoid putting hundreds of routes in one file.

app/routes/auth.py

```
from flask import Blueprint, render_template, request, redirect, url_for

auth_bp = Blueprint("auth", __name__, template_folder="templates/auth")

@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        # handle login logic
        return redirect(url_for("main.home"))
    return render_template("auth/login.html")

@auth_bp.route("/logout")
def logout():
    return redirect(url_for("auth.login"))
```

`app/routes/main.py`

```
from flask import Blueprint, render_template

main_bp = Blueprint("main", __name__)

@main_bp.route("/")
def home():
    return render_template("main/home.html")
```

Registering blueprints in `app/__init__.py`

```
from app.routes.auth import auth_bp
from app.routes.main import main_bp

app.register_blueprint(auth_bp)
app.register_blueprint(main_bp)
```

Key point about `url_for()` with blueprints

Once routes are inside a blueprint, `url_for()` needs the blueprint name prefixed:

```
url_for("auth.login")     # not url_for("login")
url_for("main.home")      # not url_for("home")
```

---

7. Where Configuration and Secrets Go

`app/config.py`

```
import os

class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY", "fallback-dev-key")
    SQLALCHEMY_DATABASE_URI = os.environ.get("DATABASE_URL", "sqlite:///app.db")
    DEBUG = False

class DevelopmentConfig(Config):
    DEBUG = True

class ProductionConfig(Config):
    DEBUG = False
```

`.env` **(never commit this file to Git)**

```
SECRET_KEY=super-secret-production-key
DATABASE_URL=postgresql://user:pass@localhost/mydb
```

`.gitignore` **— always exclude these**

```
venv/
.env
__pycache__/
*.pyc
instance/
```

---

8. Comparison of the Three Levels

Aspect	Level 1 (Single file)	Level 2 (Small multi-page)	Level 3 (Factory + Blueprints)
Best for	Learning, tiny demos	Small real projects	Team projects, larger apps
Routes location	One `app.py`	`app.py` or a `routes.py`	Split across `blueprints`
Templates	Flat `templates/` folder	Flat, maybe with `base.html`	Organized into subfolders per feature
Config	Inline in `app.py`	Separate `config.py`	Config classes + `.env`
Scalability	Poor beyond ~5 routes	Good up to a moderate size	Scales to large codebases

---

9. Your Current Project — What to Do Next

Based on what you've built so far (login, registration, profile, Jinja practice), you're at the **Level 1 → Level 2 transition point**. A reasonable next step, without jumping straight to blueprints:

```
Python_with_flask/
│
├── venv/
├── app.py
├── config.py
├── requirements.txt
│
├── templates/
│   ├── base.html
│   ├── login.html
│   ├── register.html
│   ├── profile.html
│   └── form_handling/
│       ├── register.html
│       ├── success.html
│       └── registration_base.html
│
└── static/
    ├── css/
    └── js/
```

You can move to Level 3 (application factory + blueprints) once you have several distinct feature areas (e.g. auth, blog, dashboard) that each deserve their own file.

---

10. Quick Reference Cheat Sheet

```
run.py                  → starts the app
app/__init__.py          → create_app() factory
app/config.py             → settings
app/models/                → database tables
app/routes/                  → blueprints (grouped routes)
app/templates/                → HTML files
app/static/                    → CSS, JS, images
.env                             → secrets (never commit)
requirements.txt                  → pip freeze > requirements.txt
```

11. Golden Rules

1. `templates/` and `static/` must always keep those exact names — Flask looks for them automatically.
2. Don't put secrets (API keys, DB passwords) directly in code — use `.env` and read via `os.environ`.
3. Move to blueprints once a single routes file starts feeling cluttered (rule of thumb: more than ~6-8 routes, or multiple distinct feature areas).
4. Always add `venv/` and `.env` to `.gitignore` before pushing to GitHub.
5. Keep templates organized in subfolders that mirror your blueprint structure once the project grows (`templates/auth/` , `templates/main/` , etc.).